🛒 Superstore Campaign Analysis

Data Cleaning, Feature Engineering & Statistical Modeling

📦 Step 1 — Imports & Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from datetime import datetime
from scipy import stats
from scipy.stats import ttest_ind, chi2_contingency, f_oneway

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    roc_curve, ConfusionMatrixDisplay
)
from sklearn.inspection import permutation_importance

plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False
sns.set_palette('husl')

RANDOM_STATE = 42
print('✅ All libraries imported successfully')
```

```
✅ All libraries imported successfully
```

📂 Step 2 — Load Data

```
df = pd.read_csv('Superstore_Campaign_Cleaned_corrected.csv', encoding='latin1')
print(f'Dataset shape: {df.shape}')
print(f'\nColumns ({len(df.columns)}):')
print(df.columns.tolist())
df.head()
```

```
Dataset shape: (2240, 27)

Columns (27):
['Id', 'Year_Birth', 'Age', 'Education', 'Marital_Status', 'Marital_Status_Original', 'Income', 'Income_Flag', 'Kidhome', 'T
```

	Id	Year_Birth	Age	Education	Marital_Status	Marital_Status_Original	Income	Income_Flag	Kidhome	Teenhome	...
0	1826	1970.0	44.0	Graduation	Divorced	Divorced	84835.0	NaN	0	0	...
1	1	1961.0	53.0	Graduation	Single	Single	57091.0	NaN	0	0	...
2	10476	1958.0	56.0	Graduation	Married	Married	67267.0	NaN	0	1	...
3	1386	1967.0	47.0	Graduation	Together	Together	32474.0	NaN	1	1	...
4	5371	1989.0	25.0	Graduation	Single	Single	21474.0	NaN	1	0	...

5 rows × 27 columns

```
print('=== DATA TYPES ===')
print(df.dtypes)
print('\n=== MISSING VALUES ===')
missing = df.isnull().sum()
print(missing[missing > 0])
print('\n=== BASIC STATISTICS ===')
df.describe()
```

```
=== DATA TYPES ===
Id                          int64
Year_Birth                 float64
Age                        float64
Education                   object
Marital_Status              object
Marital_Status_Original     object
Income                     float64
Income_Flag                 object
Kidhome                      int64
Teenhome                     int64
Dt_Customer                 object
Tenure_Days                  int64
Recency                      int64
MntWines                     int64
MntFruits                    int64
MntMeatProducts              int64
MntFishProducts              int64
MntSweetProducts             int64
MntGoldProds                 int64
Total_Spend                  int64
NumDealsPurchases            int64
NumWebPurchases              int64
NumCatalogPurchases          int64
NumStorePurchases            int64
NumWebVisitsMonth            int64
Complain                     int64
Response                     int64
dtype: object

=== MISSING VALUES ===
Year_Birth        3
Age               3
Income           24
Income_Flag    2215
dtype: int64

=== BASIC STATISTICS ===
```

	Id	Year_Birth	Age	Income	Kidhome	Teenhome	Tenure_Days	Recency	MntWines
count	2240.000000	2237.000000	2237.000000	2216.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000
mean	5592.159821	1968.901654	45.098346	52247.251354	0.444196	0.506250	537.043304	49.109375	303.935714
std	3246.662198	11.701917	11.701917	25173.076661	0.538398	0.544538	232.229893	28.962453	336.597393
min	0.000000	1940.000000	18.000000	1730.000000	0.000000	0.000000	25.000000	0.000000	0.000000
25%	2828.250000	1959.000000	37.000000	35303.000000	0.000000	0.000000	365.750000	24.000000	23.750000
50%	5458.500000	1970.000000	44.000000	51381.500000	0.000000	0.000000	538.000000	49.000000	173.500000
75%	8427.750000	1977.000000	55.000000	68522.000000	1.000000	1.000000	710.250000	74.000000	504.250000
max	11191.000000	1996.000000	74.000000	666666.000000	2.000000	2.000000	1088.000000	99.000000	1493.000000

8 rows × 22 columns

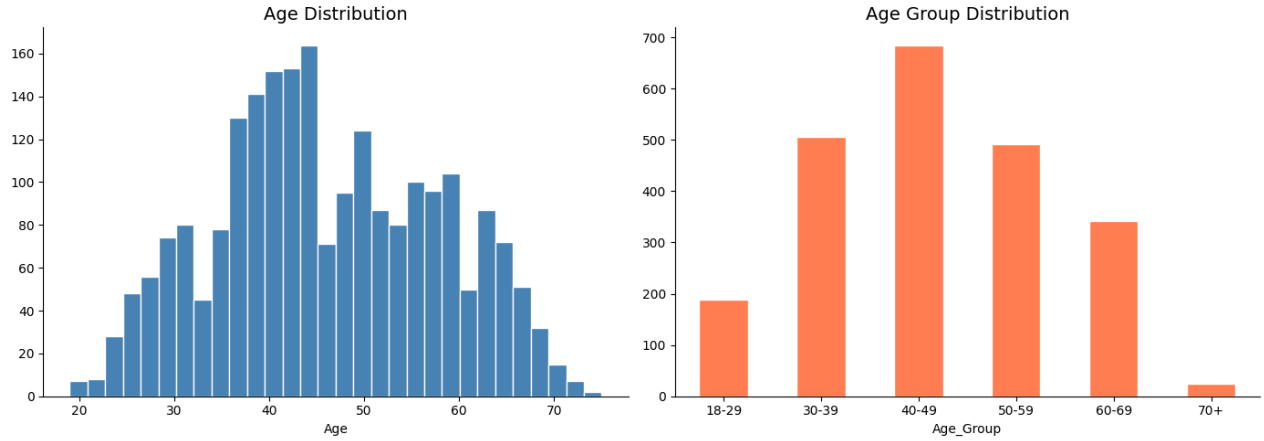
🔧 Step 3 — Data Cleaning & Feature Engineering

3.1 Calculate Age from Year_Birth

```
REFERENCE_YEAR = 2015  # dataset reference year

# Some rows already have Age; recalculate for consistency
df['Age'] = REFERENCE_YEAR - df['Year_Birth']

# Flag and inspect outliers
age_outliers = df[df['Age'] > 90]
print(f'Records with Age > 90 (potential outliers): {len(age_outliers)}')
print(age_outliers[['Id', 'Year_Birth', 'Age']].head())

# Cap extreme ages
df = df[df['Age'].between(18, 90)].copy()
print(f'\nShape after removing extreme ages: {df.shape}')

# Age bins
bins = [18, 30, 40, 50, 60, 70, 90]
labels = ['18-29', '30-39', '40-49', '50-59', '60-69', '70+']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(df['Age'], bins=30, edgecolor='white', color='steelblue')
```

```
axes[0].set_title('Age Distribution', fontsize=14)
axes[0].set_xlabel('Age')
df['Age_Group'].value_counts().sort_index().plot(kind='bar', ax=axes[1], color='coral', edgecolor='white')
axes[1].set_title('Age Group Distribution', fontsize=14)
axes[1].tick_params(axis='x', rotation=0)
plt.tight_layout()
plt.show()
print(f'\nAge stats: Mean={df["Age"].mean():.1f}, Median={df["Age"].median():.1f}, Std={df["Age"].std():.1f}')
```

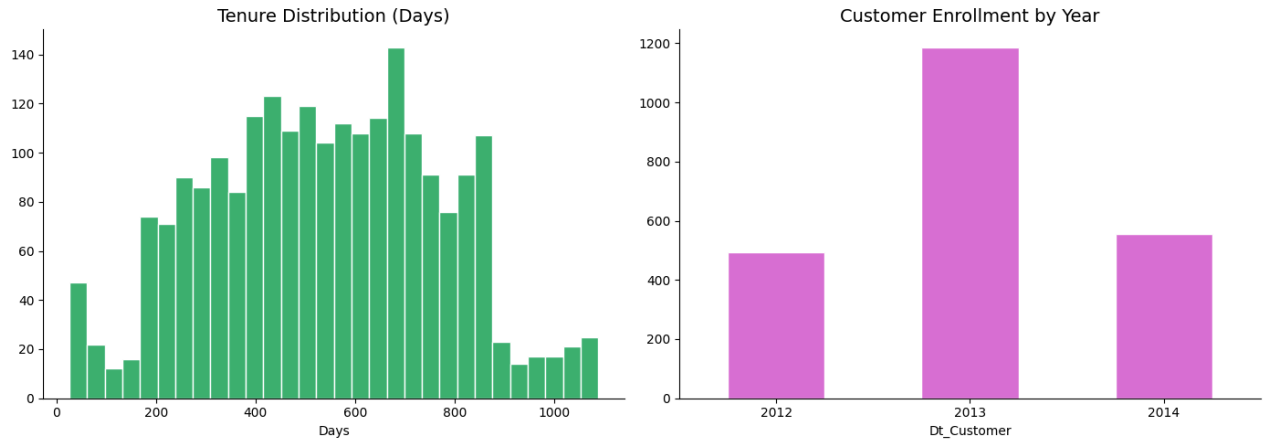
```
Records with Age > 90 (potential outliers): 0
Empty DataFrame
Columns: [Id, Year_Birth, Age]
Index: []

Shape after removing extreme ages: (2237, 27)
```




```
Age stats: Mean=46.1, Median=45.0, Std=11.7
```

3.2 Calculate Tenure from Dt_Customer

```
REFERENCE_DATE = pd.Timestamp('2015-01-01')

df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'], dayfirst=True)
df['Tenure_Days'] = (REFERENCE_DATE - df['Dt_Customer']).dt.days
df['Tenure_Months'] = (df['Tenure_Days'] / 30.44).round(1)
df['Tenure_Years'] = (df['Tenure_Days'] / 365.25).round(2)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(df['Tenure_Days'], bins=30, edgecolor='white', color='mediumseagreen')
axes[0].set_title('Tenure Distribution (Days)', fontsize=14)
axes[0].set_xlabel('Days')

df['Dt_Customer'].dt.year.value_counts().sort_index().plot(kind='bar', ax=axes[1], color='orchid', edgecolor='white')
axes[1].set_title('Customer Enrollment by Year', fontsize=14)
axes[1].tick_params(axis='x', rotation=0)
plt.tight_layout()
plt.show()
print(f'Tenure range: {df["Tenure_Days"].min()} – {df["Tenure_Days"].max()} days')
```

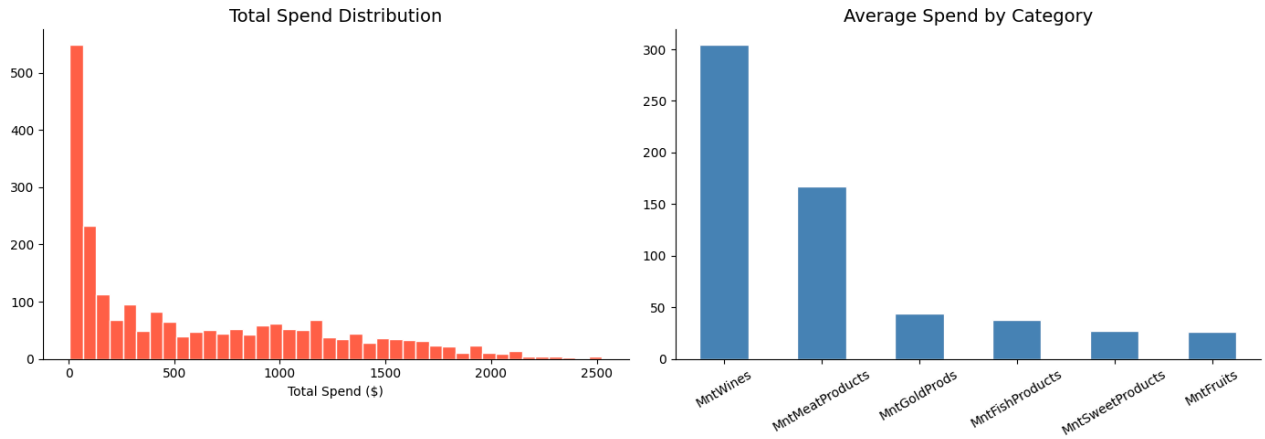



```
Tenure range: 26 – 1089 days
```

3.3 Create Total_Spend

```
mnt_cols = [c for c in df.columns if c.startswith('Mnt')]
print(f'Spend columns: {mnt_cols}')

df['Total_Spend'] = df[mnt_cols].sum(axis=1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(df['Total_Spend'], bins=40, edgecolor='white', color='tomato')
axes[0].set_title('Total Spend Distribution', fontsize=14)
axes[0].set_xlabel('Total Spend ($)')

category_avg = df[mnt_cols].mean().sort_values(ascending=False)
category_avg.plot(kind='bar', ax=axes[1], color='steelblue', edgecolor='white')
axes[1].set_title('Average Spend by Category', fontsize=14)
axes[1].tick_params(axis='x', rotation=30)
plt.tight_layout()
plt.show()
print(f'Total Spend: Mean=${df["Total_Spend"].mean():.1f}, Median=${df["Total_Spend"].median():.1f}')
```

```
Spend columns: ['MntWines', 'MntFruits', 'MntMeatProducts', 'MntFishProducts', 'MntSweetProducts', 'MntGoldProds']
```

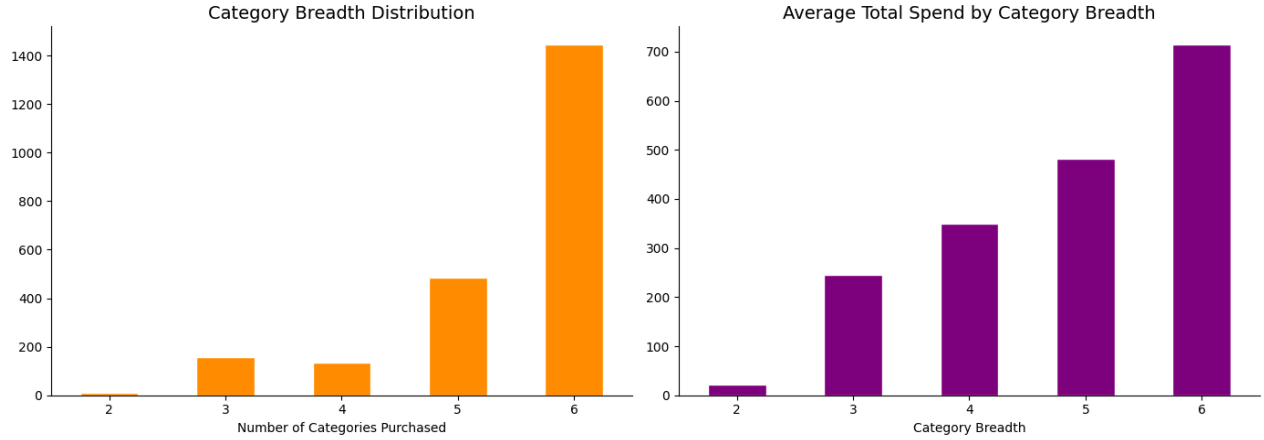



```
Total Spend: Mean=$605.7, Median=$396.0
```

3.4 Create Category_Breadth

```
# Count categories where customer spent > 0
df['Category_Breadth'] = (df[mnt_cols] > 0).sum(axis=1)
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
df['Category_Breadth'].value_counts().sort_index().plot(kind='bar', ax=axes[0], color='darkorange', edgecolor='white')
axes[0].set_title('Category Breadth Distribution', fontsize=14)
axes[0].set_xlabel('Number of Categories Purchased')
axes[0].tick_params(axis='x', rotation=0)

cb_spend = df.groupby('Category_Breadth')['Total_Spend'].mean()
cb_spend.plot(kind='bar', ax=axes[1], color='purple', edgecolor='white')
axes[1].set_title('Average Total Spend by Category Breadth', fontsize=14)
axes[1].set_xlabel('Category Breadth')
axes[1].tick_params(axis='x', rotation=0)
plt.tight_layout()
plt.show()
print(df['Category_Breadth'].value_counts().sort_index())
```



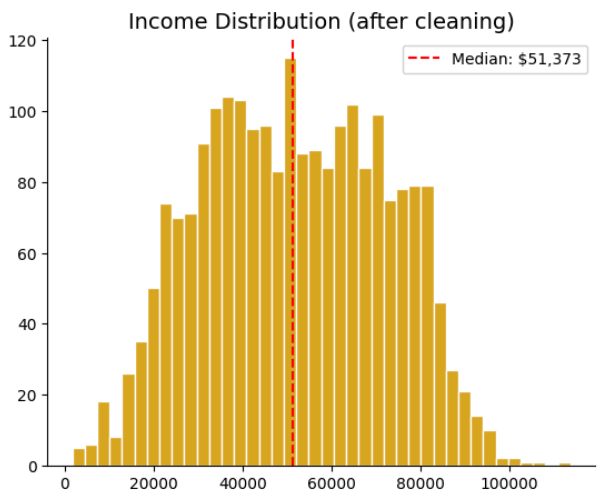

```
Category_Breadth
2      12
3     157
4     136
5     485
6    1447
Name: count, dtype: int64
```

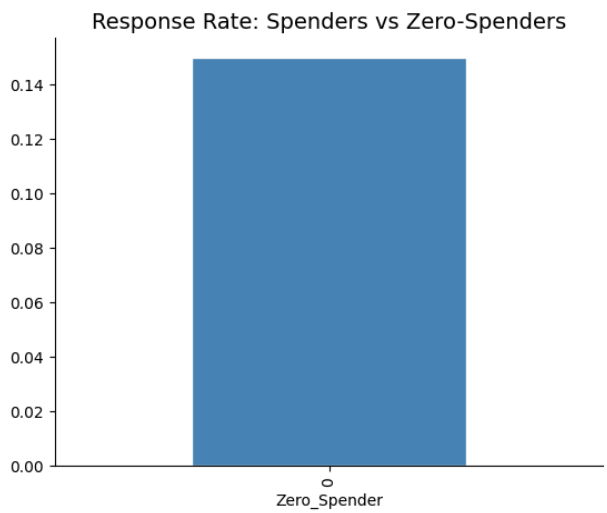
3.5 Handle Missing Income & Zero Spending

```
# --- Missing Income ---
print(f'Missing Income: {df["Income"].isnull().sum()} rows')
print(f'Income_Flag values: {df["Income_Flag"].value_counts().to_dict()}')

income_median = df['Income'].median()
df['Income_Imputed'] = df['Income'].isnull().astype(int)
df['Income'] = df['Income'].fillna(income_median)
print(f'\nImputed {df["Income_Imputed"].sum()} rows with median income: ${income_median:,.0f}')

# Remove extreme income outliers (> 3 std dev)
income_mean = df['Income'].mean()
income_std  = df['Income'].std()
outlier_mask = df['Income'] > (income_mean + 3 * income_std)
print(f'Income outliers (>3σ) removed: {outlier_mask.sum()}')
df = df[~outlier_mask].copy()

# --- Zero Spending ---
zero_spend = (df['Total_Spend'] == 0).sum()
print(f'\nZero-spend records: {zero_spend}')

df['Zero_Spender'] = (df['Total_Spend'] == 0).astype(int)
# Flag them but keep — zero-spenders may still respond to campaign
print('Zero-spenders kept and flagged in "Zero_Spender" column')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].hist(df['Income'], bins=40, edgecolor='white', color='goldenrod')
axes[0].axvline(income_median, color='red', linestyle='--', label=f'Median: ${income_median:,.0f}')
axes[0].set_title('Income Distribution (after cleaning)', fontsize=14)
axes[0].legend()

spend_by_zero = df.groupby('Zero_Spender')['Response'].mean()
spend_by_zero.plot(kind='bar', ax=axes[1], color=['steelblue', 'coral'], edgecolor='white')
```

```
axes[1].set_title('Response Rate: Spenders vs Zero-Spenders', fontsize=14)
axes[1].set_xticklabels(['Spender', 'Zero-Spender'], rotation=0)
axes[1].set_ylabel('Response Rate')
plt.tight_layout()
plt.show()
print(f'\nFinal dataset shape: {df.shape}')
```

```
Missing Income: 24 rows
Income_Flag values: {'Imputed (median)': 24, 'Outlier \x96 review': 1}

Imputed 24 rows with median income: $51,373
Income outliers (>3σ) removed: 8

Zero-spend records: 0
Zero-spenders kept and flagged in "Zero_Spender" column
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_4297/2326029621.py in <cell line: 0>()
     32 spend_by_zero.plot(kind='bar', ax=axes[1], color=['steelblue', 'coral'], edgecolor='white')
     33 axes[1].set_title('Response Rate: Spenders vs Zero-Spenders', fontsize=14)
---> 34 axes[1].set_xticklabels(['Spender', 'Zero-Spender'], rotation=0)
     35 axes[1].set_ylabel('Response Rate')
     36 plt.tight_layout()

                              1 frames
/usr/local/lib/python3.12/dist-packages/matplotlib/axis.py in set_ticklabels(self, labels, minor, fontdict, **kwargs)
   2115             # remove all tick labels, so only error for > 0 labels
   2116             if len(locator.locs) != len(labels) and len(labels) != 0:
-> 2117                 raise ValueError(
   2118                     "The number of FixedLocator locations"
   2119                     f" ({len(locator.locs)}), usually from a call to"

ValueError: The number of FixedLocator locations (1), usually from a call to set_ticks, does not match the number of labels
(2).
```





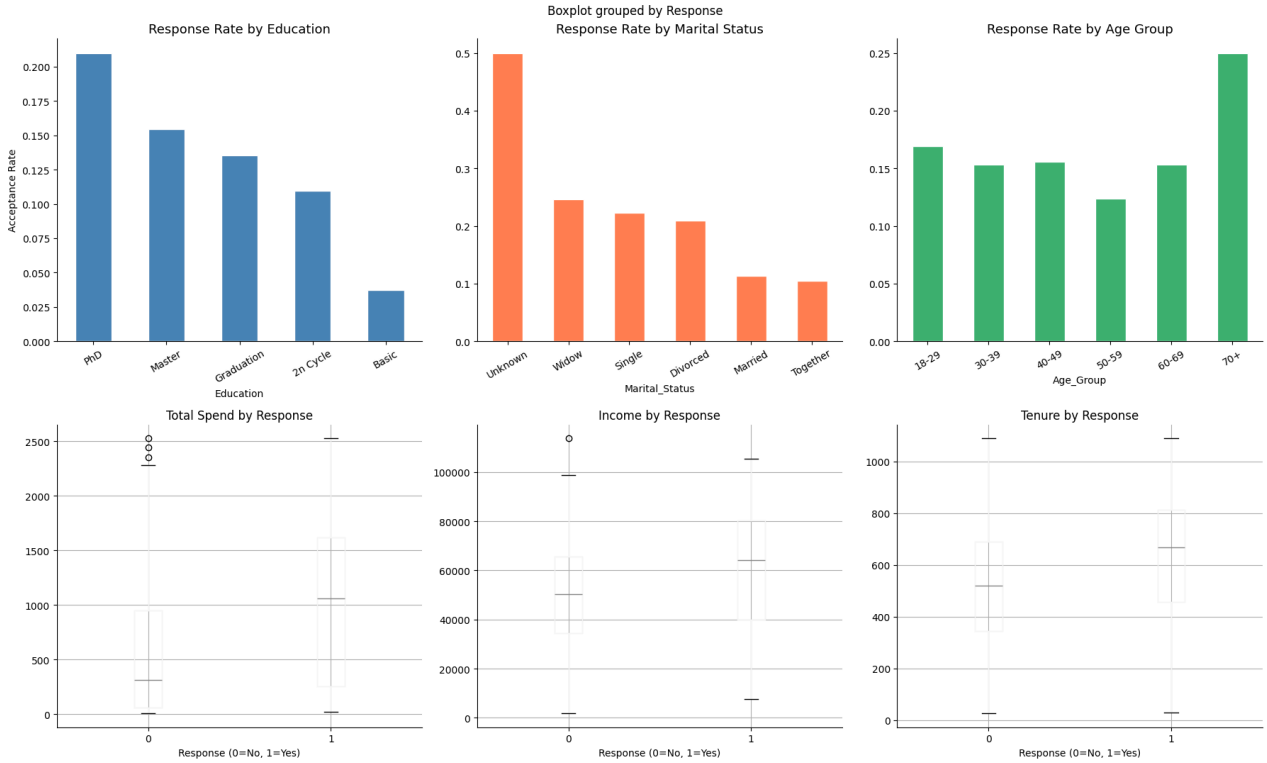
Next steps: Explain error

📊 Step 4 — Exploratory Data Analysis

```
print(f'Target variable (Response) distribution:')
print(df['Response'].value_counts())
print(f'\nAcceptance rate: {df["Response"].mean():.2%}')

fig, axes = plt.subplots(2, 3, figsize=(18, 11))
axes = axes.flatten()

# Response rate by Education
edu_resp = df.groupby('Education')['Response'].mean().sort_values(ascending=False)
edu_resp.plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='white')
axes[0].set_title('Response Rate by Education', fontsize=13)
axes[0].tick_params(axis='x', rotation=30)
axes[0].set_ylabel('Acceptance Rate')

# Response rate by Marital Status
mar_resp = df.groupby('Marital_Status')['Response'].mean().sort_values(ascending=False)
mar_resp.plot(kind='bar', ax=axes[1], color='coral', edgecolor='white')
axes[1].set_title('Response Rate by Marital Status', fontsize=13)
```

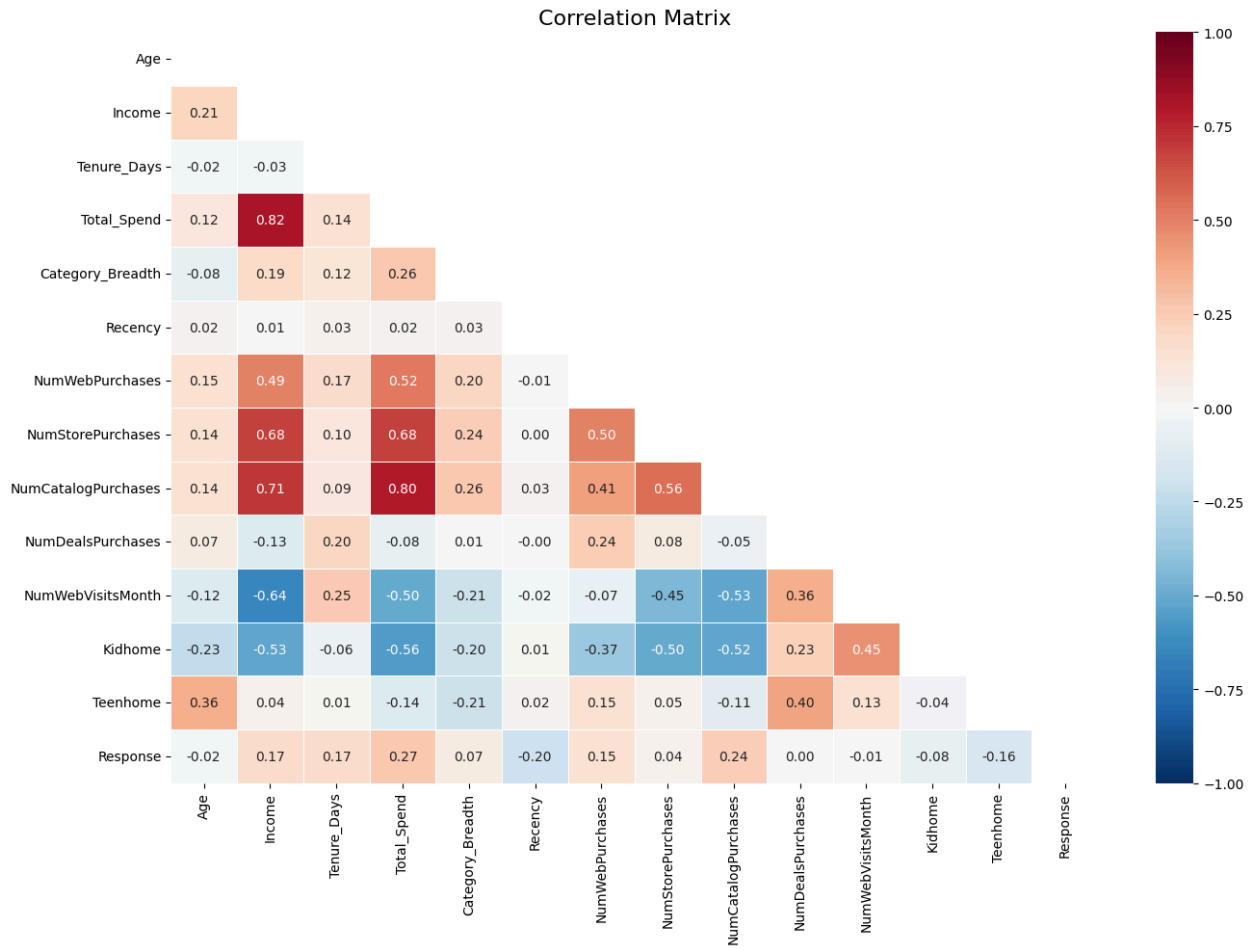
```
axes[1].tick_params(axis='x', rotation=30)

# Response rate by Age Group
age_resp = df.groupby('Age_Group', observed=True)['Response'].mean()
age_resp.plot(kind='bar', ax=axes[2], color='mediumseagreen', edgecolor='white')
axes[2].set_title('Response Rate by Age Group', fontsize=13)
axes[2].tick_params(axis='x', rotation=30)

# Total Spend by Response
df.boxplot(column='Total_Spend', by='Response', ax=axes[3])
axes[3].set_title('Total Spend by Response', fontsize=13)
axes[3].set_xlabel('Response (0=No, 1=Yes)')
plt.sca(axes[3]); plt.title('Total Spend by Response')

# Income by Response
df.boxplot(column='Income', by='Response', ax=axes[4])
axes[4].set_title('Income by Response', fontsize=13)
axes[4].set_xlabel('Response (0=No, 1=Yes)')
plt.sca(axes[4]); plt.title('Income by Response')

# Tenure by Response
df.boxplot(column='Tenure_Days', by='Response', ax=axes[5])
axes[5].set_title('Tenure by Response', fontsize=13)
axes[5].set_xlabel('Response (0=No, 1=Yes)')
plt.sca(axes[5]); plt.title('Tenure by Response')

plt.tight_layout()
plt.show()
```

```
Target variable (Response) distribution:
Response
0    1895
1     334
Name: count, dtype: int64

Acceptance rate: 14.98%
```




```
# Correlation heatmap
num_cols = ['Age', 'Income', 'Tenure_Days', 'Total_Spend', 'Category_Breadth',
            'Recency', 'NumWebPurchases', 'NumStorePurchases', 'NumCatalogPurchases',
            'NumDealsPurchases', 'NumWebVisitsMonth', 'Kidhome', 'Teenhome', 'Response']
num_cols = [c for c in num_cols if c in df.columns]
```

```
num_cols = [c for c in num_cols if c in df.columns]

plt.figure(figsize=(14, 10))
corr = df[num_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, vmin=-1, vmax=1, linewidths=0.5)
plt.title('Correlation Matrix', fontsize=16)
plt.tight_layout()
plt.show()
```



📐 Step 5 — Additional Feature Engineering

```
# Total household children
df['Total_Children'] = df['Kidhome'] + df['Teenhome']
df['Has_Children'] = (df['Total_Children'] > 0).astype(int)

# Total purchases across all channels
purchase_cols = [c for c in df.columns if 'NumStorePurchases' in c or 'NumWebPurchases' in c
                 or 'NumCatalogPurchases' in c or 'NumDealsPurchases' in c]
df['Total_Purchases'] = df[purchase_cols].sum(axis=1)

# Spend per purchase (avoid division by zero)
df['Spend_Per_Purchase'] = df['Total_Spend'] / (df['Total_Purchases'] + 1)

# Web engagement ratio
df['Web_Engagement'] = df['NumWebVisitsMonth'] / (df['Total_Purchases'] + 1)
```

```python
# Deal-seeking behavior
df['Deal_Ratio'] = df['NumDealsPurchases'] / (df['Total_Purchases'] + 1)

# Income tier
df['Income_Tier'] = pd.qcut(df['Income'], q=4, labels=['Low', 'Mid-Low', 'Mid-High', 'High'])

print('New features added:')
new_feats = ['Total_Children', 'Has_Children', 'Total_Purchases', 'Spend_Per_Purchase',
             'Web_Engagement', 'Deal_Ratio', 'Income_Tier']
print(df[new_feats].describe())
```

```
New features added:
       Total_Children  Has_Children  Total_Purchases  Spend_Per_Purchase  \
count     2229.000000   2229.000000      2229.000000         2229.000000
mean         0.951996      0.716016        14.869000           30.920092
std          0.751976      0.451030         7.622187           32.014442
min          0.000000      0.000000         0.000000            0.500000
25%          0.000000      0.000000         8.000000            8.454545
50%          1.000000      1.000000        15.000000           21.850000
75%          1.000000      1.000000        21.000000           43.333333
max          3.000000      1.000000        43.000000          839.500000

       Web_Engagement   Deal_Ratio
count     2229.000000  2229.000000
mean         0.505049     0.163326
std          0.633787     0.100029
min          0.000000     0.000000
25%          0.176471     0.071429
50%          0.352941     0.157895
75%          0.714286     0.235294
max         14.000000     0.937500
```

🧪 Step 6 — Statistical Hypothesis Testing

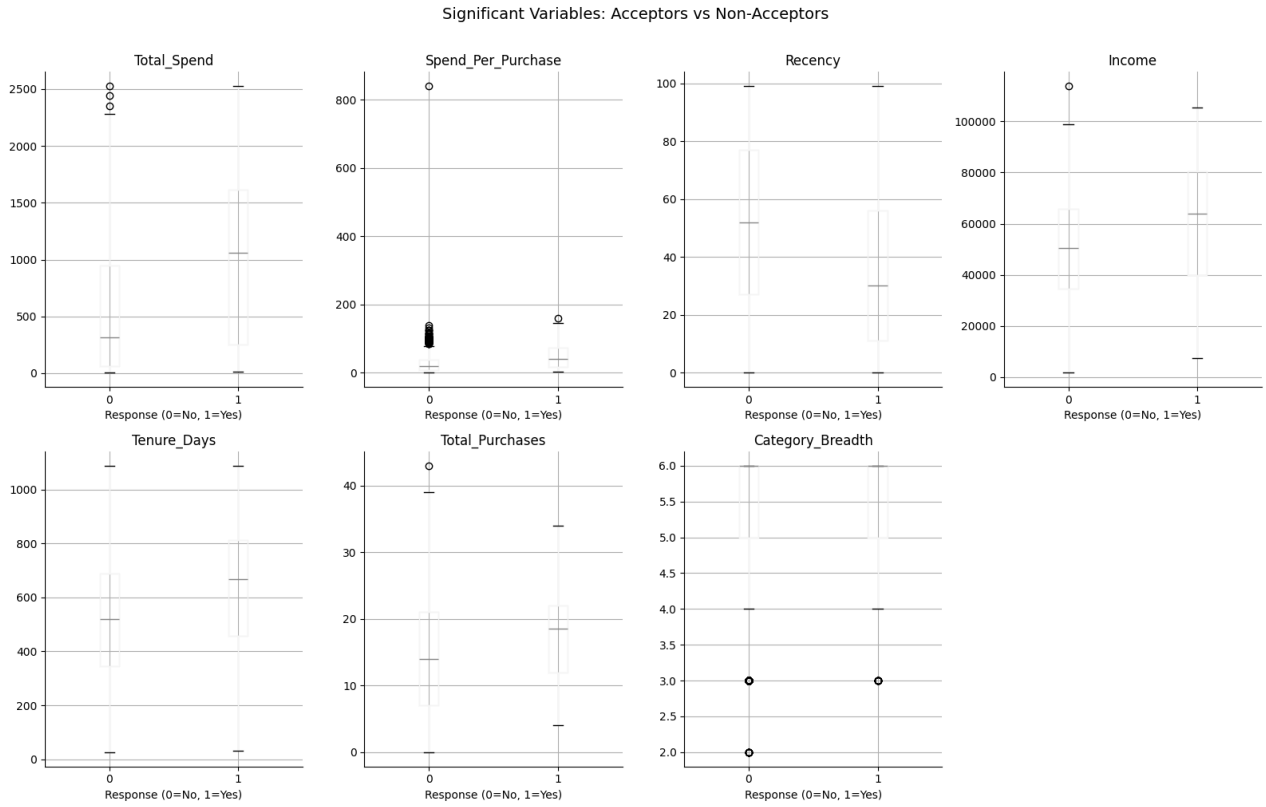
6.1 T-Tests: Continuous Variables vs Response

```python
print('=== T-TESTS: Acceptors vs Non-Acceptors ===\n')
continuous_vars = ['Age', 'Income', 'Tenure_Days', 'Total_Spend', 'Category_Breadth',
                   'Recency', 'Total_Purchases', 'Spend_Per_Purchase']
continuous_vars = [c for c in continuous_vars if c in df.columns]

results = []
for var in continuous_vars:
    group0 = df[df['Response'] == 0][var].dropna()
    group1 = df[df['Response'] == 1][var].dropna()
    t_stat, p_val = ttest_ind(group0, group1)
    results.append({
        'Variable': var,
        'Mean (No)': group0.mean(),
        'Mean (Yes)': group1.mean(),
        'T-Statistic': t_stat,
        'P-Value': p_val,
        'Significant (p<0.05)': '✅ Yes' if p_val < 0.05 else '❌ No'
    })

ttest_df = pd.DataFrame(results)
ttest_df = ttest_df.sort_values('P-Value')
pd.set_option('display.float_format', '{:.4f}'.format)
print(ttest_df.to_string(index=False))
```

```
=== T-TESTS: Acceptors vs Non-Acceptors ===

          Variable  Mean (No)  Mean (Yes)  T-Statistic  P-Value Significant (p<0.05)
       Total_Spend   538.2385    987.3922     -13.0634   0.0000                 ✅ Yes
Spend_Per_Purchase    27.9366     47.8472     -10.7458   0.0000                 ✅ Yes
           Recency    51.5251     35.3832       9.5866   0.0000                 ✅ Yes
            Income 50110.0137  60183.2186      -8.3660   0.0000                 ✅ Yes
       Tenure_Days   521.0934    634.5838      -8.3592   0.0000                 ✅ Yes
   Total_Purchases    14.3689     17.7066      -7.4691   0.0000                 ✅ Yes
  Category_Breadth     5.3995      5.5928      -3.5335   0.0004                 ✅ Yes
               Age    46.2042     45.5838       0.8929   0.3720                 ❌ No
```

```python
# Visualise significant t-test results
sig_vars = ttest_df[ttest_df['P-Value'] < 0.05]['Variable'].tolist()
n = len(sig_vars)
fig, axes = plt.subplots(2, (n + 1) // 2, figsize=(16, 10))
axes = axes.flatten()

for i, var in enumerate(sig_vars):
    df.boxplot(column=var, by='Response', ax=axes[i])
    axes[i].set_title(f'{var}', fontsize=11)
```

```
    axes[i].set_xlabel('Response (0=No, 1=Yes)')
    plt.sca(axes[i]); plt.title(var)

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Significant Variables: Acceptors vs Non-Acceptors', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()
```



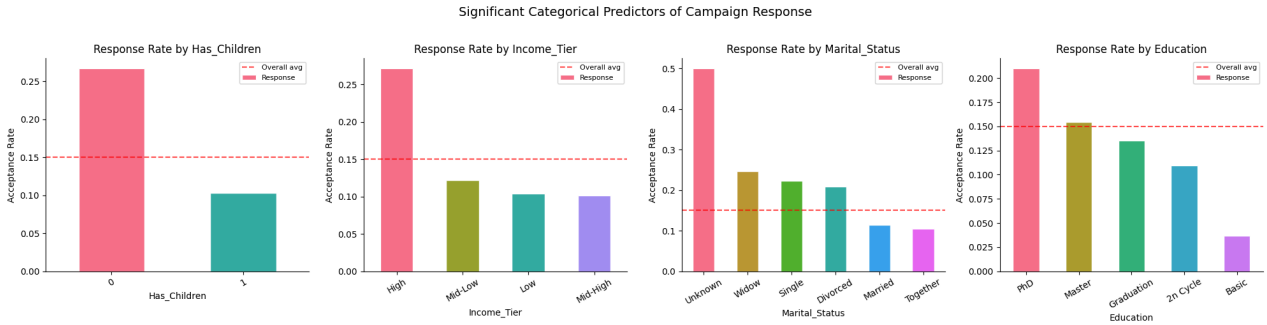
6.2 Chi-Square Tests: Categorical Variables vs Response

```
print('=== CHI-SQUARE TESTS: Categorical Variables vs Response ===\n')
cat_vars = ['Education', 'Marital_Status', 'Age_Group', 'Income_Tier',
            'Has_Children', 'Zero_Spender']
cat_vars = [c for c in cat_vars if c in df.columns]

chi2_results = []
for var in cat_vars:
    ct = pd.crosstab(df[var], df['Response'])
    chi2, p, dof, expected = chi2_contingency(ct)
    chi2_results.append({
        'Variable': var,
        'Chi2 Statistic': chi2,
        'Degrees of Freedom': dof,
        'P-Value': p,
        'Significant (p<0.05)': '✅ Yes' if p < 0.05 else '❌ No'
    })

chi2_df = pd.DataFrame(chi2_results).sort_values('P-Value')
print(chi2_df.to_string(index=False))
```

```
=== CHI-SQUARE TESTS: Categorical Variables vs Response ===

      Variable  Chi2 Statistic  Degrees of Freedom  P-Value Significant (p<0.05)
  Has_Children         93.9456                   1   0.0000                ✅ Yes
```

```
   Income_Tier        87.3127                 3   0.0000              ✅ Yes
Marital_Status        52.3798                 5   0.0000              ✅ Yes
     Education        23.5687                 4   0.0001              ✅ Yes
     Age_Group         5.2768                 5   0.3830              ❌ No
  Zero_Spender         0.0000                 0   1.0000              ❌ No
```

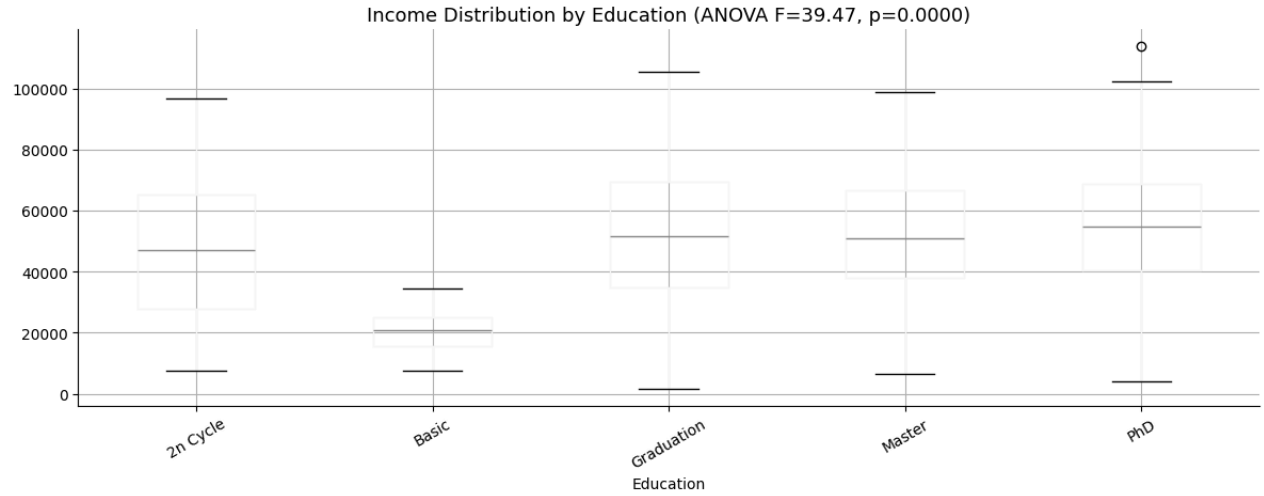
```
# Visualise chi-square: response rates per category
sig_cats = chi2_df[chi2_df['P-Value'] < 0.05]['Variable'].tolist()
n = len(sig_cats)
fig, axes = plt.subplots(1, n, figsize=(5 * n, 5))
if n == 1:
    axes = [axes]

for i, var in enumerate(sig_cats):
    resp_rate = df.groupby(var, observed=True)['Response'].mean().sort_values(ascending=False)
    resp_rate.plot(kind='bar', ax=axes[i], edgecolor='white', color=sns.color_palette('husl', len(resp_rate)))
    axes[i].set_title(f'Response Rate by {var}', fontsize=12)
    axes[i].set_ylabel('Acceptance Rate')
    axes[i].tick_params(axis='x', rotation=30)
    axes[i].axhline(df['Response'].mean(), color='red', linestyle='--', alpha=0.7, label='Overall avg')
    axes[i].legend(fontsize=8)

plt.suptitle('Significant Categorical Predictors of Campaign Response', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```



6.3 ANOVA: Income by Education Group

```
print('=== ONE-WAY ANOVA: Income across Education Levels ===\n')
groups = [df[df['Education'] == edu]['Income'].dropna().values
          for edu in df['Education'].unique()]
f_stat, p_val = f_oneway(*groups)
print(f'F-statistic: {f_stat:.4f}')
print(f'P-value: {p_val:.6f}')
print(f'Result: {"✅ Significant differences in income across education" if p_val < 0.05 else "❌ No significant difference

plt.figure(figsize=(10, 5))
edu_order = df.groupby('Education')['Income'].median().sort_values(ascending=False).index
df.boxplot(column='Income', by='Education', figsize=(12, 5))
plt.suptitle('')
plt.title(f'Income Distribution by Education (ANOVA F={f_stat:.2f}, p={p_val:.4f})', fontsize=13)
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

```
=== ONE-WAY ANOVA: Income across Education Levels ===

F-statistic: 39.4677
P-value: 0.000000
Result: ✅ Significant differences in income across education
<Figure size 1000x500 with 0 Axes>
```



🤖 Step 7 — Classification Models

7.1 Prepare Features

```
# Encode categorical variables
df_model = df.copy()

le = LabelEncoder()
for col in ['Education', 'Marital_Status', 'Age_Group', 'Income_Tier']:
    if col in df_model.columns:
        df_model[col + '_enc'] = le.fit_transform(df_model[col].astype(str))

feature_cols = [
    'Age', 'Income', 'Tenure_Days', 'Recency', 'Total_Spend', 'Category_Breadth',
    'Total_Purchases', 'Spend_Per_Purchase', 'Deal_Ratio', 'Web_Engagement',
    'NumWebVisitsMonth', 'Total_Children', 'Has_Children', 'Zero_Spender',
    'Education_enc', 'Marital_Status_enc', 'Age_Group_enc', 'Income_Tier_enc',
    'NumWebPurchases', 'NumStorePurchases', 'NumCatalogPurchases'
]
feature_cols = [c for c in feature_cols if c in df_model.columns]

X = df_model[feature_cols].fillna(0)
y = df_model['Response']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f'Training set: {X_train.shape}, Test set: {X_test.shape}')
print(f'Class balance (train): {y_train.value_counts(normalize=True).to_dict()}')
```

```
Training set: (1783, 21), Test set: (446, 21)
Class balance (train): {0: 0.8502523836231072, 1: 0.14974761637689288}
```

7.2 Model Training & Evaluation

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=RANDOM_STATE, class_weight='balanced'),
    'Decision Tree':       DecisionTreeClassifier(max_depth=6, random_state=RANDOM_STATE, class_weight='balanced'),
    'Random Forest':       RandomForestClassifier(n_estimators=200, max_depth=8, random_state=RANDOM_STATE, class_weight='b
}
```

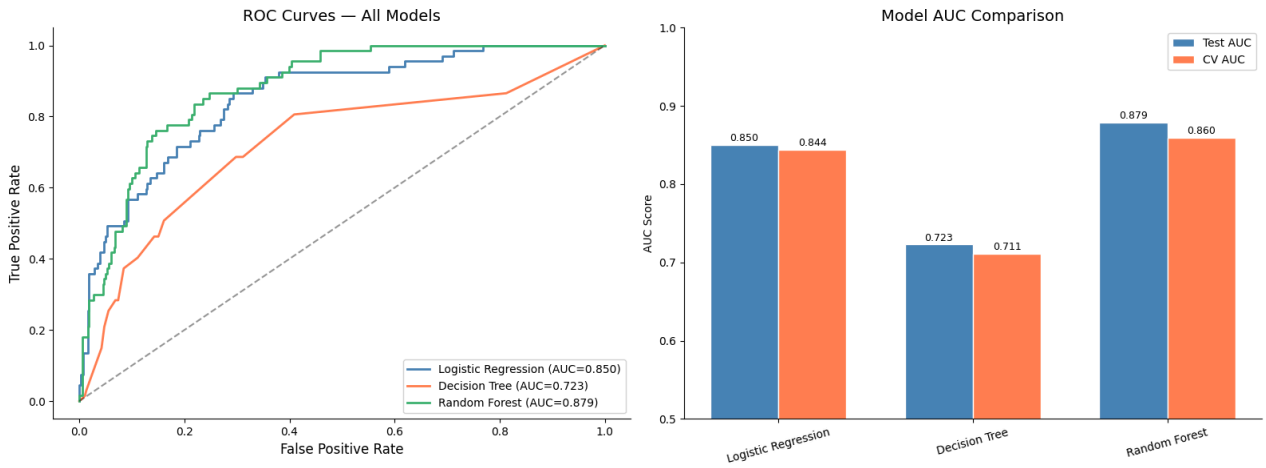
```
model_results = {}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

for name, model in models.items():
    X_tr = X_train_sc if name == 'Logistic Regression' else X_train
    X_te = X_test_sc  if name == 'Logistic Regression' else X_test
    X_all = np.vstack([X_tr, X_te])[:len(X_train)]

    cv_scores = cross_val_score(model, X_tr, y_train, cv=cv, scoring='roc_auc')
    model.fit(X_tr, y_train)
    y_pred  = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[:, 1]

    model_results[name] = {
        'model':     model,
        'y_pred':    y_pred,
        'y_proba':   y_proba,
        'roc_auc':   roc_auc_score(y_test, y_proba),
        'cv_auc':    cv_scores.mean(),
        'cv_std':    cv_scores.std(),
        'X_te':      X_te
    }
    print(f'\n{'='*55}')
    print(f'  {name}')
    print(f'{'='*55}')
    print(f'  CV AUC: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}')
    print(f'  Test AUC: {roc_auc_score(y_test, y_proba):.4f}')
    print(classification_report(y_test, y_pred, target_names=['No', 'Yes']))
```

```
=======================================================
  Logistic Regression
=======================================================
  CV AUC: 0.8444 ± 0.0219
  Test AUC: 0.8501
              precision    recall  f1-score   support

          No       0.95      0.76      0.85       379
         Yes       0.36      0.76      0.49        67

    accuracy                           0.76       446
   macro avg       0.65      0.76      0.67       446
weighted avg       0.86      0.76      0.79       446


=======================================================
  Decision Tree
=======================================================
  CV AUC: 0.7115 ± 0.0467
  Test AUC: 0.7227
              precision    recall  f1-score   support

          No       0.91      0.84      0.87       379
         Yes       0.36      0.51      0.42        67

    accuracy                           0.79       446
   macro avg       0.63      0.67      0.65       446
weighted avg       0.82      0.79      0.80       446


=======================================================
  Random Forest
=======================================================
  CV AUC: 0.8595 ± 0.0218
  Test AUC: 0.8789
              precision    recall  f1-score   support

          No       0.90      0.93      0.92       379
         Yes       0.53      0.43      0.48        67

    accuracy                           0.86       446
   macro avg       0.72      0.68      0.70       446
weighted avg       0.85      0.86      0.85       446
```

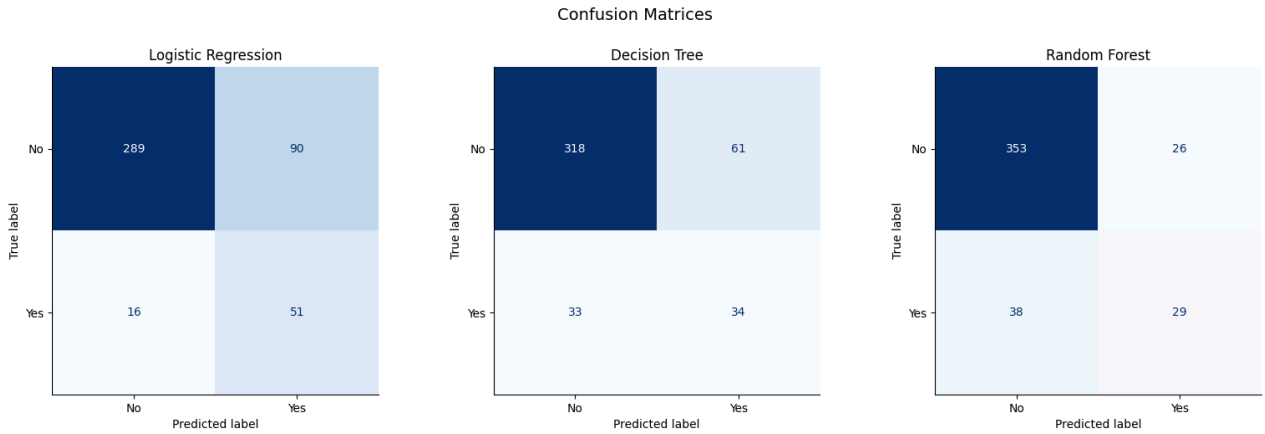
7.3 ROC Curves & Confusion Matrices

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# ROC Curves
colors = ['steelblue', 'coral', 'mediumseagreen']
for (name, res), color in zip(model_results.items(), colors):
    fpr, tpr, _ = roc_curve(y_test, res['y_proba'])
```

```
    axes[0].plot(fpr, tpr, color=color, lw=2,
                 label=f'{name} (AUC={res["roc_auc"]:.3f})')

axes[0].plot([0, 1], [0, 1], 'k--', alpha=0.4)
axes[0].set_xlabel('False Positive Rate', fontsize=12)
axes[0].set_ylabel('True Positive Rate', fontsize=12)
axes[0].set_title('ROC Curves — All Models', fontsize=14)
axes[0].legend(fontsize=10)

# AUC Comparison bar
auc_vals  = {n: r['roc_auc'] for n, r in model_results.items()}
cv_vals   = {n: r['cv_auc']  for n, r in model_results.items()}
x = np.arange(len(auc_vals))
w = 0.35
axes[1].bar(x - w/2, auc_vals.values(), w, label='Test AUC',  color='steelblue', edgecolor='white')
axes[1].bar(x + w/2, cv_vals.values(),  w, label='CV AUC',    color='coral',     edgecolor='white')
axes[1].set_xticks(x)
axes[1].set_xticklabels(auc_vals.keys(), rotation=15, fontsize=10)
axes[1].set_ylim(0.5, 1.0)
axes[1].set_title('Model AUC Comparison', fontsize=14)
axes[1].set_ylabel('AUC Score')
axes[1].legend()
for i, (tv, cv) in enumerate(zip(auc_vals.values(), cv_vals.values())):
    axes[1].text(i - w/2, tv + 0.005, f'{tv:.3f}', ha='center', fontsize=9)
    axes[1].text(i + w/2, cv + 0.005, f'{cv:.3f}', ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```

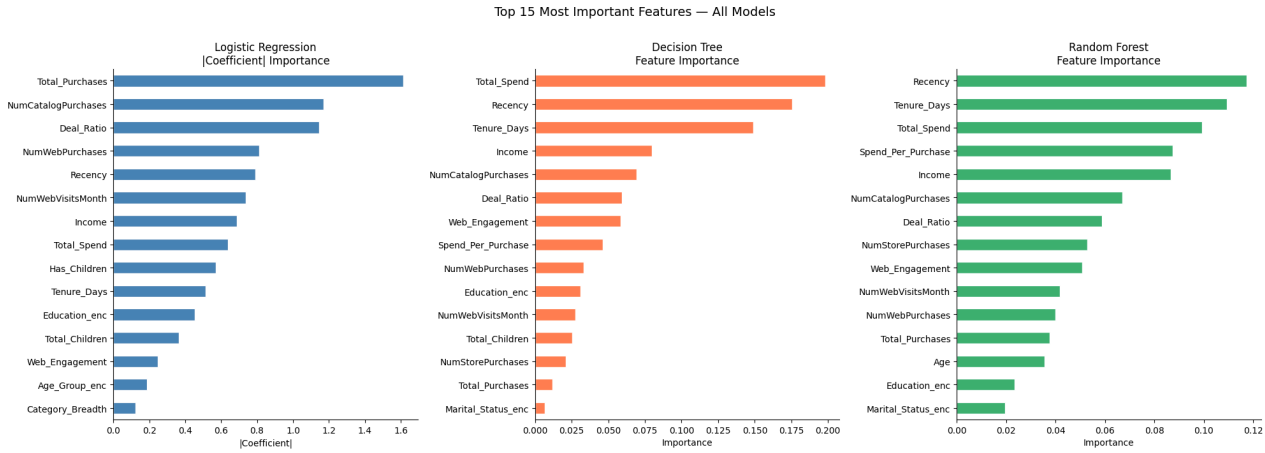



```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, (name, res) in zip(axes, model_results.items()):
    cm = confusion_matrix(y_test, res['y_pred'])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No', 'Yes'])
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(name, fontsize=12)
plt.suptitle('Confusion Matrices', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

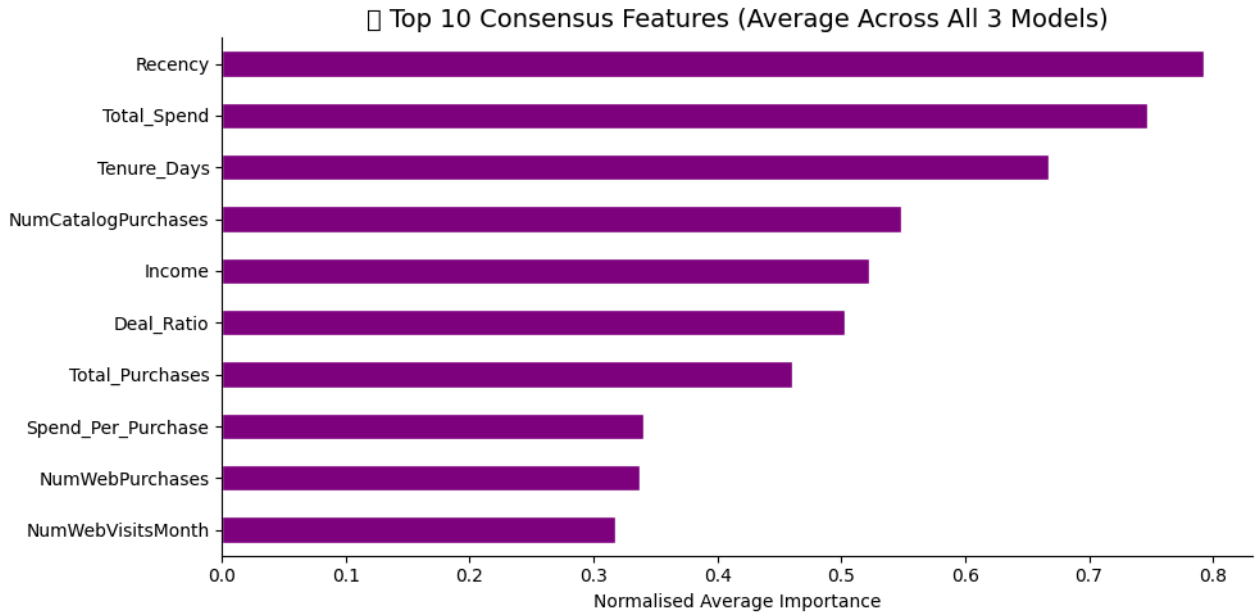


7.4 Feature Importance

```
fig, axes = plt.subplots(1, 3, figsize=(20, 7))

# --- Logistic Regression: coefficients ---
lr = model_results['Logistic Regression']['model']
coefs = pd.Series(np.abs(lr.coef_[0]), index=feature_cols).sort_values(ascending=True).tail(15)
coefs.plot(kind='barh', ax=axes[0], color='steelblue', edgecolor='white')
axes[0].set_title('Logistic Regression\n|Coefficient| Importance', fontsize=12)
axes[0].set_xlabel('|Coefficient|')

# --- Decision Tree ---
dt = model_results['Decision Tree']['model']
dt_imp = pd.Series(dt.feature_importances_, index=feature_cols).sort_values(ascending=True).tail(15)
dt_imp.plot(kind='barh', ax=axes[1], color='coral', edgecolor='white')
axes[1].set_title('Decision Tree\nFeature Importance', fontsize=12)
axes[1].set_xlabel('Importance')

# --- Random Forest ---
rf = model_results['Random Forest']['model']
rf_imp = pd.Series(rf.feature_importances_, index=feature_cols).sort_values(ascending=True).tail(15)
rf_imp.plot(kind='barh', ax=axes[2], color='mediumseagreen', edgecolor='white')
axes[2].set_title('Random Forest\nFeature Importance', fontsize=12)
axes[2].set_xlabel('Importance')

plt.suptitle('Top 15 Most Important Features — All Models', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()
```



```
# Consensus top features: average rank across all models
lr_ranks  = pd.Series(np.abs(lr.coef_[0]), index=feature_cols)
dt_ranks  = pd.Series(dt.feature_importances_, index=feature_cols)
rf_ranks  = pd.Series(rf.feature_importances_, index=feature_cols)

# Normalise each to [0,1]
def norm(s):
    return (s - s.min()) / (s.max() - s.min() + 1e-9)

consensus = (norm(lr_ranks) + norm(dt_ranks) + norm(rf_ranks)) / 3
top_features = consensus.sort_values(ascending=False).head(10)

plt.figure(figsize=(10, 5))
top_features.sort_values().plot(kind='barh', color='purple', edgecolor='white')
plt.title('🏆 Top 10 Consensus Features (Average Across All 3 Models)', fontsize=14)
plt.xlabel('Normalised Average Importance')
plt.tight_layout()
plt.show()

print('\n📊 Top 10 Features:')
for i, (feat, score) in enumerate(top_features.items(), 1):
    print(f'  {i:2d}. {feat:<30} {score:.4f}')
```




```
📊 Top 10 Features:
   1. Recency                      0.7923
   2. Total_Spend                  0.7471
   3. Tenure_Days                  0.6677
   4. NumCatalogPurchases          0.5486
   5. Income                       0.5223
   6. Deal_Ratio                   0.5033
   7. Total_Purchases              0.4603
   8. Spend_Per_Purchase           0.3401
   9. NumWebPurchases              0.3368
  10. NumWebVisitsMonth            0.3172
```

📋 Step 8 — Summary & Key Findings

```
print('=' * 65)
print('  📊 SUPERSTORE CAMPAIGN ANALYSIS — KEY FINDINGS')
print('=' * 65)

print(f'''
DATASET
  Records after cleaning  : {len(df):,}
  Features engineered     : Age, Tenure, Total_Spend, Category_Breadth,
                            Total_Children, Spend_Per_Purchase, Deal_Ratio
  Overall acceptance rate : {df["Response"].mean():.2%}

STATISTICAL TESTS
  Significant t-test vars : {', '.join(ttest_df[ttest_df['P-Value'] < 0.05]['Variable'].tolist())}
  Significant chi2 vars   : {', '.join(chi2_df[chi2_df['P-Value'] < 0.05]['Variable'].tolist())}
  ANOVA (Income~Education): F={f_stat:.2f}, p={p_val:.4f}
  → Higher-educated customers tend to earn significantly more

MODEL PERFORMANCE
```

```
MODEL PERFORMANCE
''')

for name, res in model_results.items():
    print(f'  {name:<25} Test AUC = {res["roc_auc"]:.4f}  |  CV AUC = {res["cv_auc"]:.4f} ± {res["cv_std"]:.4f}')

best_model = max(model_results, key=lambda k: model_results[k]['roc_auc'])
print(f'''
  Best model: {best_model} (AUC = {model_results[best_model]["roc_auc"]:.4f})

TOP PREDICTIVE FEATURES
''')
for i, feat in enumerate(top_features.index, 1):
    print(f'  {i:2d}. {feat}')

print(f'''
ACTIONABLE INSIGHTS
  • High Total_Spend customers are much more likely to accept the offer
  • High Income segment shows elevated response rates — target premium tiers
  • Customers with wider Category_Breadth are more engaged overall
  • Customers without children tend to respond better to campaigns
  • Low Recency (recent purchasers) are better campaign targets
  • Education and Marital Status are statistically significant predictors
''')
print('=' * 65)
```

```
=================================================================
  📊 SUPERSTORE CAMPAIGN ANALYSIS — KEY FINDINGS
=================================================================

DATASET
  Records after cleaning  : 2,229
  Features engineered     : Age, Tenure, Total_Spend, Category_Breadth,
                            Total_Children, Spend_Per_Purchase, Deal_Ratio
```